NTI HireReady Scholarship — DEY Program

Project Documentation — Architecture, Design, and Golden Model

Direct-Mapped Cache Controller with an AHB-Lite Interface

Team: Ahmed Tawfik – Antonios Adel – Manar Tarek

Under Supervision of Eng. Mohamed Salah

Contents

1.Introduction

1.1. Project Overview

A cache is a small and fast memory placed between a processor and a larger but slower main memory. Its purpose is to reduce the average memory access time by storing recently accessed data close to the processor.

This project implements a ***Direct-Mapped Cache Controller with an AMBA AHB-Lite interface***. The cache controller manages communication between a CPU-side AHB-Lite interface and a memory-side AHB-Lite interface.

The implemented cache contains ***64 cache lines***, with each line containing ***4 words***. Since each word is 32 bits, each cache line contains:

[4 = 128 bit] or 16 bytes.

The controller is divided into three main functional blocks:

1. ***CPU-Side Slave FSM***
2. ***Memory Side Master FSM***
3. ***Cache Core***

The CPU-Side Slave FSM receives requests from the processor and performs cache accesses. If a cache miss occurs, it stalls the processor and activates the Memory-Side Master FSM.

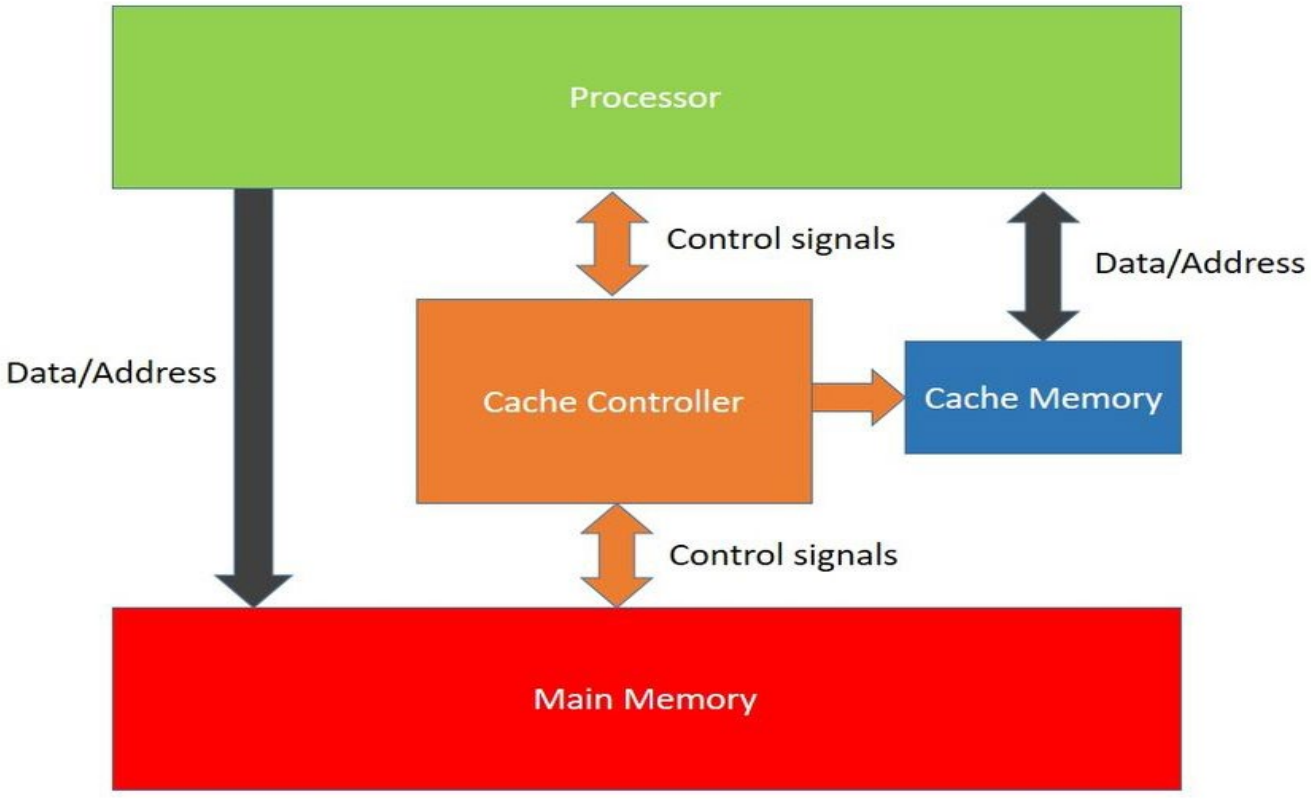
The Memory-Side Master FSM communicates with main memory through AHB-Lite. During a read miss, it performs a four-word burst transfer and sends the received data to the Cache Core.

The Cache Core contains the actual cache storage, including the tag array, valid bits, data array, and line buffer

1.2. Project Objectives

The main objectives of the project are:

- Design a functional direct-mapped cache.
- Implement a 64-line cache organization.
- Store four 32-bit words in every cache line.
- Support CPU read and write requests.
- Detect cache hits and misses.
- Implement a write-through policy.
- Handle cache line refill after read misses.
- Use AHB-Lite for communication with main memory.
- Implement an INCR4 burst transfer for line refill.
- Correctly handle AHB-Lite wait states using HREADY.
- Design the memory-side controller using an FSM.
- Verify the design using simulation and testbenches.



1.3. Project Scope

The implemented system focuses on a simple single-level direct-mapped cache. The main features included in the scope are:

- 32-bit address space 64 cache lines
- 4 words per line
- 32-bit word size
- Direct mapping
- Write-through operation
- AHB-Lite CPU-side interface
- AHB-Lite memory-side interface
- Four-word INCR4 cache refill
- Valid-bit-based cache line management
- Tag comparison
- Read and write transaction handling
- HREADY-based wait-state handling

Advanced features such as multi-level caches, associative mapping, replacement algorithms, speculative access, and non-blocking cache operation are outside the current scope Considered as a Future work.

2.System Specifications

2.1. Cache Specs

Parameter	Value
Cache organization	Direct-mapped, single level
Total capacity	1 KB
Number of lines	64
Line size	4 words = 16 bytes
Write policy	Write-through, with cache line updated on write-hit
Address breakdown	Tag[31:10] (22 bits) | Index[9:4] (6 bits) | Word offset[3:2] (2 bits) | Byte offset[1:0] (unused, word-aligned)
CPU-side interface	AHB-Lite slave
Memory-side interface	AHB-Lite master

2.2 Memory Specifications

The main memory is accessed using a 32-bit AHB-Lite data bus.

The memory-side controller generates:

- Address
- Transfer type
- Read/write control
- Transfer size
- Burst type
- Write data

For read misses, the controller retrieves an entire cache line rather than only the requested word.

For example, if the requested address is: 0x0000100C

The corresponding cache line starts at: 0x00001000

The four memory words are:

0x00001000
0x00001004
0x00001008
0x0000100C

2.3 AHB-Lite Interface Specifications

1. Core Signal List

Signal	Driven By	Width	Meaning
HCLK	System	1	Bus clock. All signals are sampled on the rising edge.
HRESETn	System	1	Active-low reset.
HADDR	Master	32	Address for the current transfer (address phase).
HTRANS	Master	2	Transfer type: IDLE / BUSY / NONSEQ / SEQ (see §2).
HWRITE	Master	1	1 = write, 0 = read.
HSIZE	Master	3	Transfer size: byte / halfword / word (see §3).
HBURST	Master	3	Burst type: single, INCR, WRAP4/8/16, INCR4/8/16 (see §4).
HWDATA	Master	32	Write data, valid in the data phase of a write.
HRDATA	Slave	32	Read data, valid in the data phase of a read.
HREADY	Slave	1	1 = transfer finished this cycle. 0 = slave inserts a wait state.
HRESP	Slave	1	OKAY or ERROR (see §5).
HSEL	Decoder	1	Selects this slave for the current address phase.

2. HTRANS — Transfer Type

Value	Name	Meaning
2'b00	IDLE	No transfer requested. Ignore HWRITE/HADDR this cycle.
2'b01	BUSY	Master inserts a gap mid-burst; not needed for your first pass — you can treat as unused.
2'b10	NONSEQ	First transfer of a burst, or a single standalone transfer. This is what you'll use most.
2'b11	SEQ	Subsequent transfer within a burst — address is the previous address + size.

3. HSIZE — Transfer Size

Value	Meaning
3'b000	Byte (8 bits)
3'b001	Halfword (16 bits)
3'b010	Word (32 bits) — use this for your design

4. HBURST — Burst Type

Value	Name	Meaning
3'b000	SINGLE	Single transfer, not part of a burst.
3'b001	INCR	Incrementing burst, undefined length.
3'b011	INCR4	4-beat incrementing burst — recommended for your line-fill if your cache line = 4 words.
3'b101	INCR8	8-beat incrementing burst — use if your line size = 8 words.
3'b010 / 3'b100 / 3'b110	WRAP4 / WRAP8 / WRAP16	Wrapping bursts — not needed for a straightforward line-fill; skip for v1.

5. HRESP — Slave Response

Value	Name	Meaning
1'b0	OKAY	Transfer completed normally. This is the only response your backing memory model needs to generate.
1'b1	ERROR	Transfer failed. Not needed for v1 — your memory model can always return OKAY.

word transfer: **HSIZE = 3'b010**

For a four-beat incrementing burst:

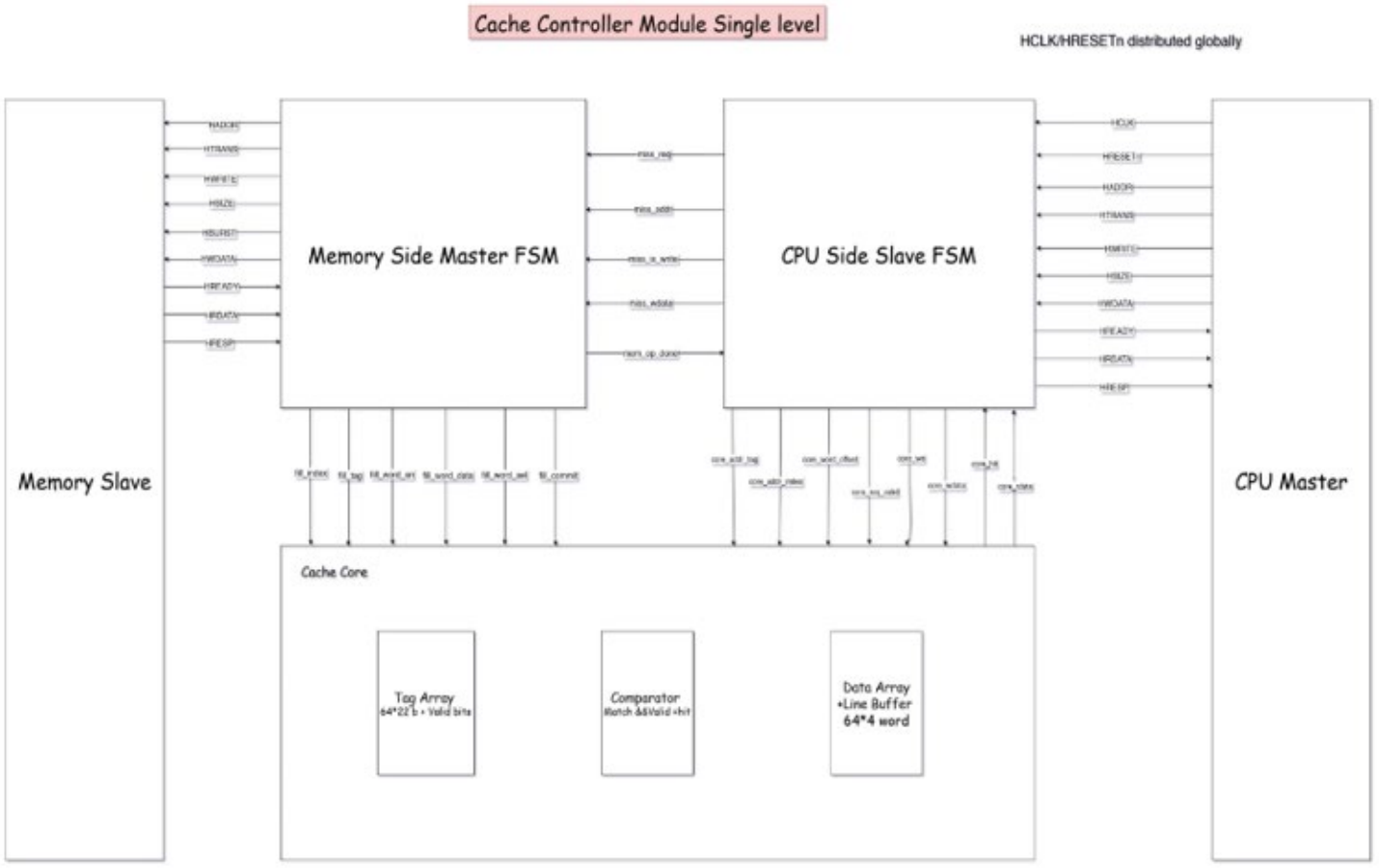
HBURST = 3'b011

3. System Architecture

The system sits between two AHB-Lite buses. On the CPU side, the cache controller behaves as a slave: it accepts requests, decides hit or miss, and stalls the bus (HREADY = 0) while a miss is serviced. On the memory side, the cache controller behaves as a master: it generates its own AHB-Lite transfers to read or write backing memory, entirely independently of CPU bus timing.

Internally, the design is split into two control planes (the two finite state machines) and one storage/decision plane (the cache core), connected by two simple internal buses: the “request/response” bus between the CPU-side FSM and the cache core, and the “fill” bus between the memory-side FSM and the cache core.

3.1 Block Diagram



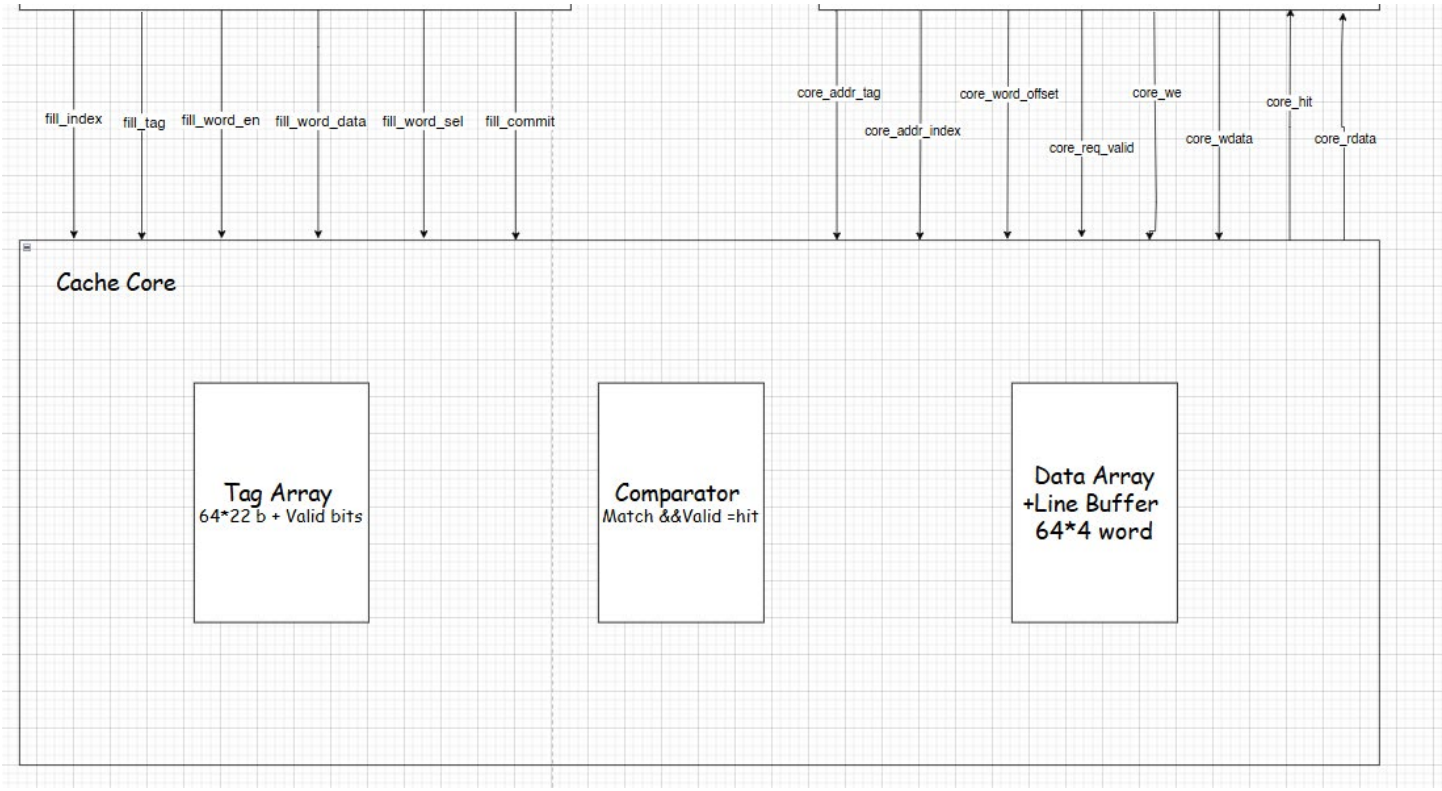
3.2 Signal Groups Between Blocks

Bus	Between	Purpose
CPU AHB-Lite slave port	External master ↔ cpu_side_slave_fsm	Standard AHB-Lite: HADDR, HTRANS, HWRITE, HSIZE, HBURST, HWDATA, HREADY, HRDATA, HRESP
Core request/response bus	cpu_side_slave_fsm ↔ cache_core	core_addr_tag, core_addr_index, core_word_offset, core_req_valid, core_we, core_wdata → ; core_hit, core_rdata ←
Miss-service bus	cpu_side_slave_fsm ↔ mem_side_master_fsm	miss_req, miss_addr, miss_is_write, miss_wdata → ; mem_op_done ←
Fill bus	mem_side_master_fsm → cache_core	fill_index, fill_tag, fill_word_en, fill_word_sel, fill_word_data, fill_commit
Memory AHB-Lite master port	mem_side_master_fsm ↔ external backing memory	Standard AHB-Lite, this block as master

4. Block-by-Block Explanation

This section explains every RTL block in plain language: what it is, what problem it solves, and how it behaves. Blocks are presented in the order data flows through the system.

4.1 tag_valid_array.v — “Does this cache line belong to this address?”



This block is the cache's memory of which address each of the 64 lines currently holds. For every line it stores two things: a 22-bit tag (the upper address bits identifying which memory block is cached) and a 1-bit valid flag (whether that line's contents are meaningful yet, or still garbage from before reset).

Reading is combinational and instant: give it an index, it immediately outputs the stored tag and valid bit for that line, with no clock delay. Writing only happens on a “fill commit” pulse from the memory-side FSM, at which point the new tag is stored and the valid bit is set to 1. On reset, all valid bits are cleared to 0 so that nothing appears to be a hit before any real data has been loaded — this is what makes the very first access to any line correctly report a miss.

4.2 comparator.v — “Is this a hit?”

```
module comparator (
    input  [21:0] addr_tag,
    input  [21:0] stored_tag,
    input         valid,
    input         req_valid,
    output        hit
);

  assign hit = req_valid && (addr_tag == stored_tag) && valid;

endmodule
```

The comparator is the single point of truth for hit/miss decisions. It is pure combinational logic with no memory of its own: a hit is declared only if the request is actually valid, the incoming address tag exactly matches the tag stored for that index, and the line's valid bit is set.

The valid-bit check is deliberately included even though it looks redundant — without it, an all-zero tag on an all-zero, freshly-reset line would falsely look like a match, since 0 equals 0. This exact scenario (tag = 0, index = 0, right after reset) is called out specifically in the cache-core testbench as Test 1, precisely because it is the case that would silently pass if this bit were left out.

4.3 data_line_array.v — “Where the actual cached data lives”

This block holds the real cached data: 64 lines, each 128 bits (four 32-bit words) wide. It has three separate jobs, all coexisting in the same small module:

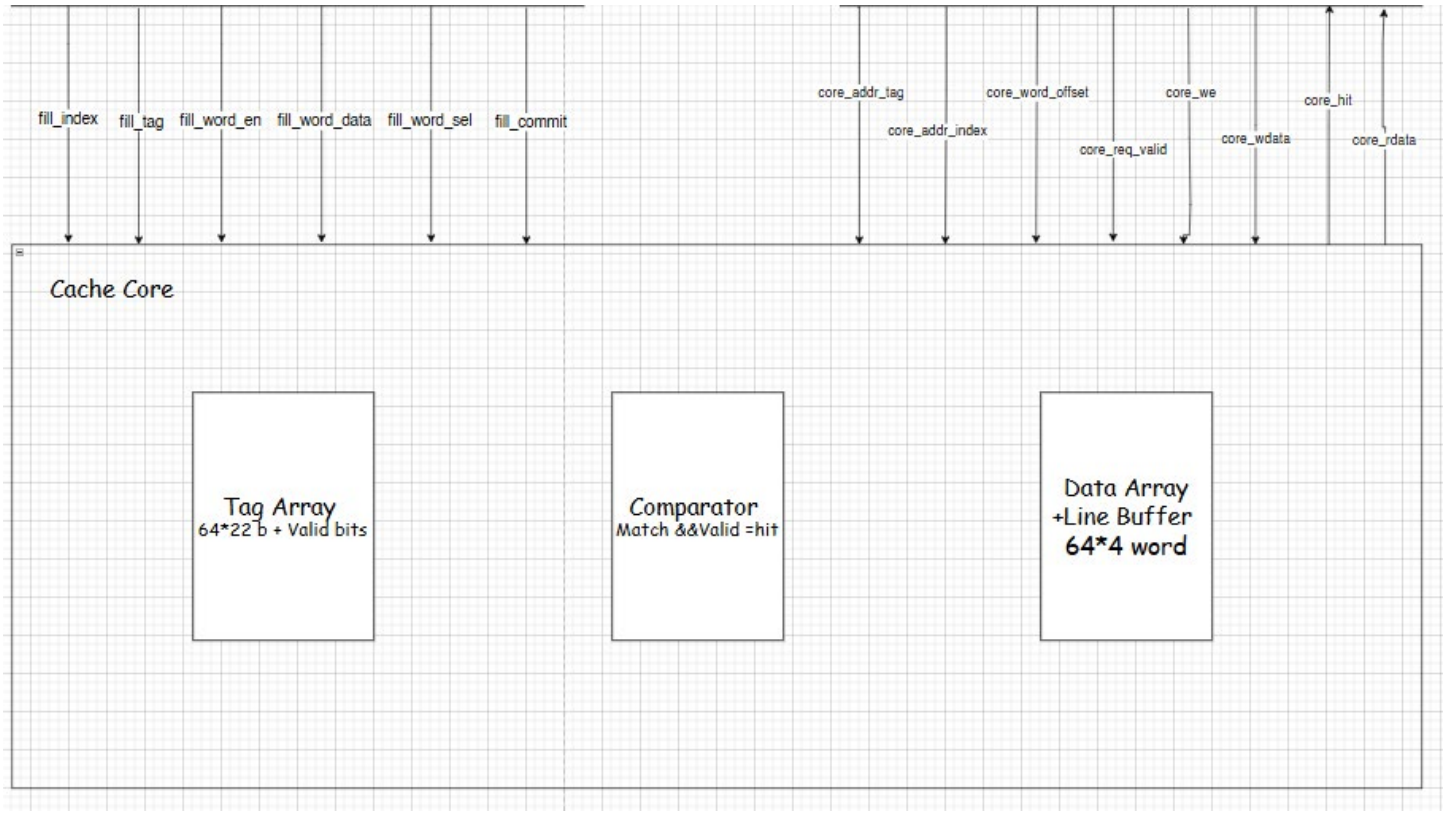
- Combinational read: given an index and a word offset (0–3), it returns the correct 32-bit word instantly, with no clock delay — this is what makes cache hits fast.
- Write-hit update: on a clock edge, if a write-hit is signaled, it updates only the one 32-bit word that was written, leaving the other three words in that line completely untouched (partial-word write).
- Line fill: incoming words from a memory read are first accumulated one at a time into a temporary 128-bit staging buffer (as they arrive over the AHB burst), and only once all four words have arrived does a single “commit” pulse write the entire assembled line into the data array in one atomic step.

The two-phase fill design (buffer, then atomic commit) exists so that a line is never left half-written if something were to interrupt the fill sequence — the data array only ever sees a complete, consistent line.

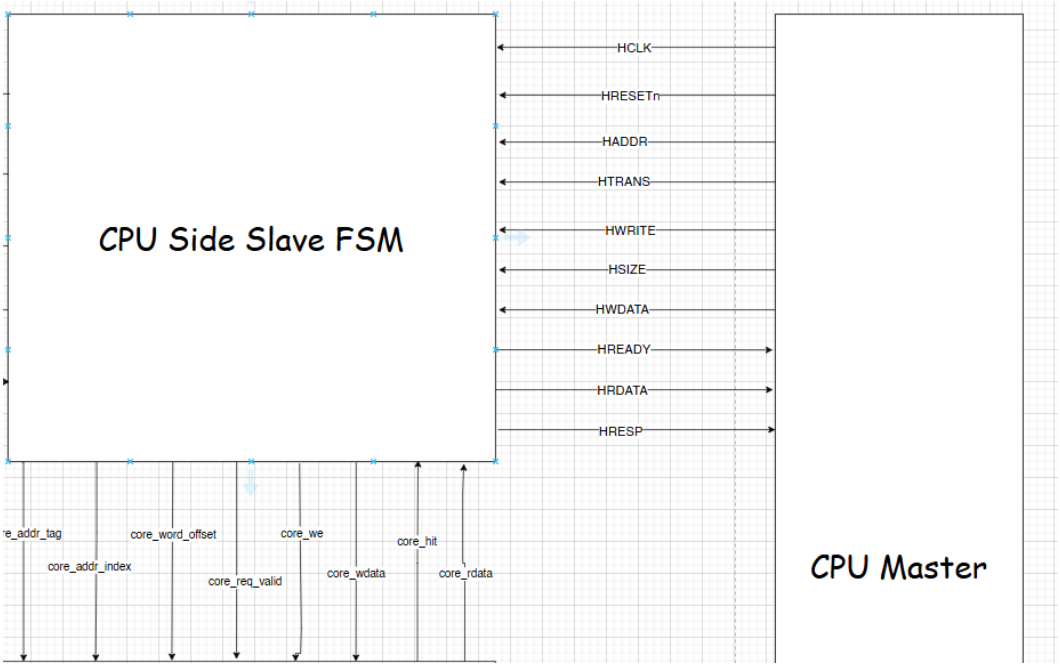
4.4 cache_core.v — “The cache itself, with no AHB awareness”

This module is simply the previous three blocks wired together into one clean, protocol-agnostic unit. It has no idea that AHB-Lite exists; it just accepts a tag/index/offset/valid/write request and produces a hit signal and read data, plus a separate fill interface for loading new lines.

One deliberate safety choice here: the write-enable signal sent into the data array is not simply passed through from the CPU-side FSM — it is re-gated inside cache_core by cache_core's own hit computation (we_gated = core_we && core_hit). This means the data array can never be corrupted by a write signal arriving on a miss, even if there were ever a bug upstream in the FSM that asserted “write” without first confirming a hit.

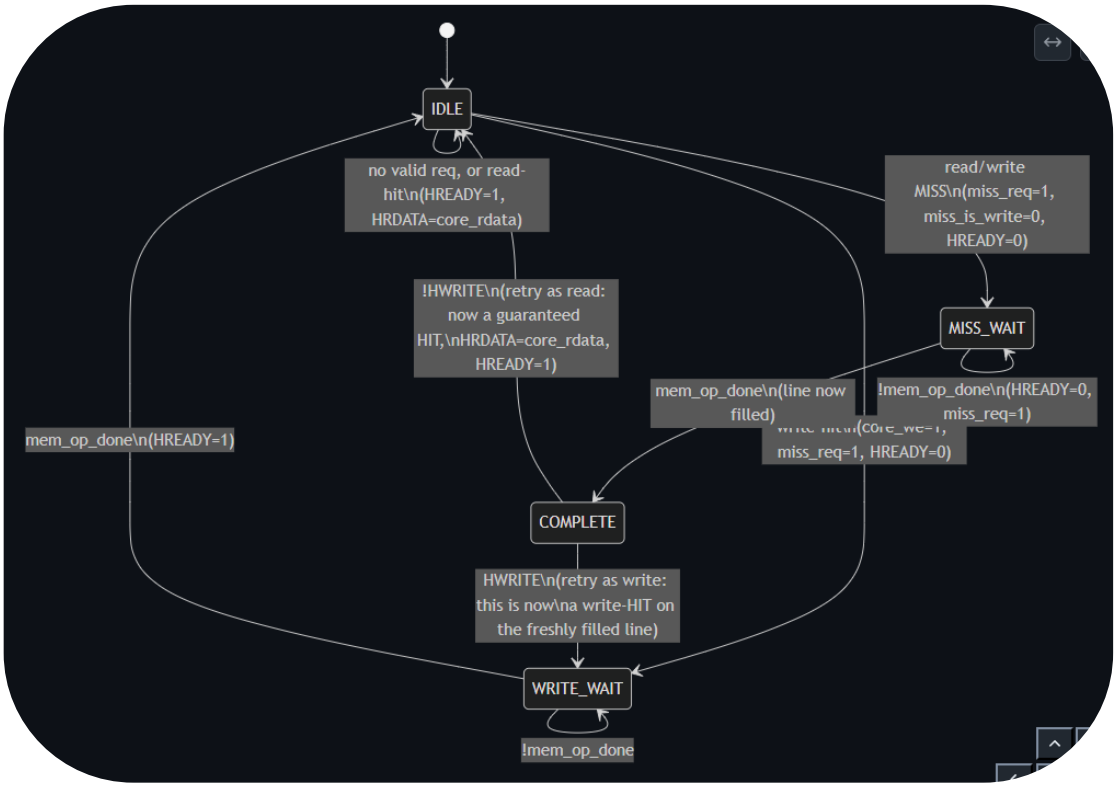


4.5 cpu_side_slave_fsm.v — "The negotiator with the CPU"



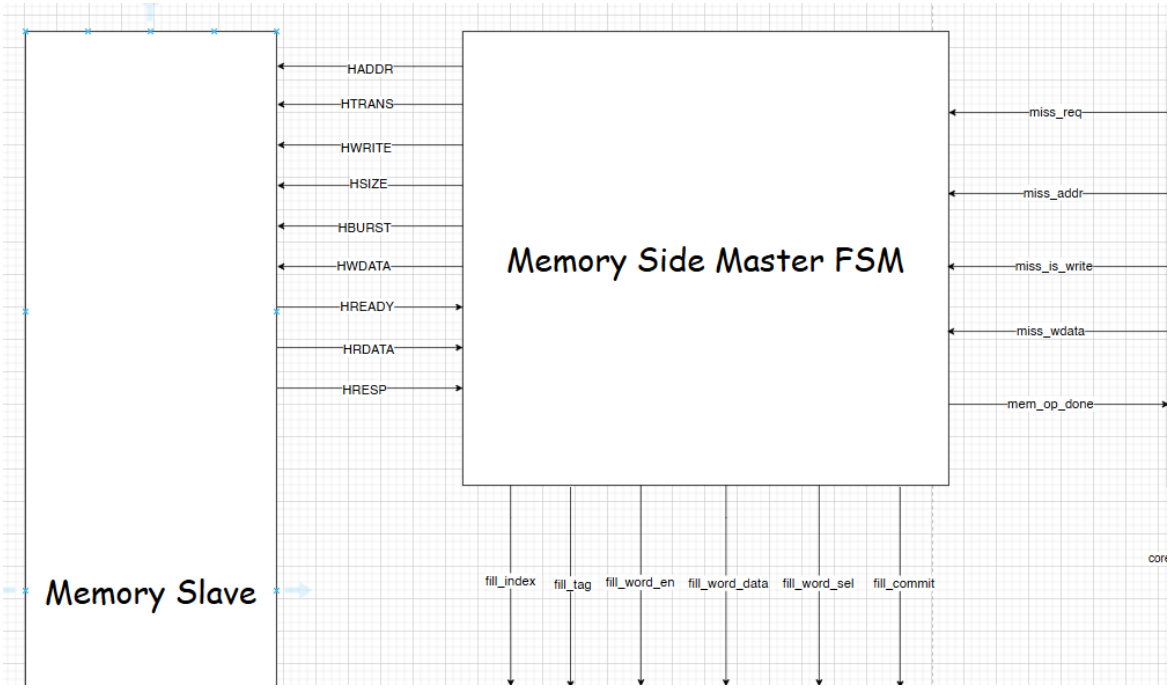
This is the AHB-Lite slave state machine that the CPU (or any AHB-Lite master) talks to directly. Its job is to make the cache look, from the CPU's point of view, like an ordinary AHB-Lite slave — hiding all the complexity of hits, misses, and memory fills behind the standard HREADY-based stall mechanism.

It breaks every incoming address into tag, index, and word offset, and immediately asks cache_core whether this is a hit. On a read-hit, it returns data in the very same cycle with no stall. On a write-hit, it still has to go to memory (write-through), so it asserts miss_req toward the memory-side FSM and stalls the bus (HREADY = 0) until that write completes. On any miss, it stalls the bus, asks the memory-side FSM to service the miss, and once mem_op_done arrives, re-issues the very same request into cache_core — which is now a guaranteed hit because the line was just filled — to actually deliver the data or perform the write.



State	Meaning
STATE_IDLE	Waiting for a new AHB request; resolves hit/miss combinationally as soon as one arrives.
STATE_MISS_WAIT	A read-miss is being serviced by the memory-side FSM; bus is stalled.
STATE_WRITE_WAIT	A write (hit or post-fill) is being written through to memory; bus is stalled.
STATE_COMPLETE	The line has just been filled; re-issue the original request into cache_core to finish the transaction.

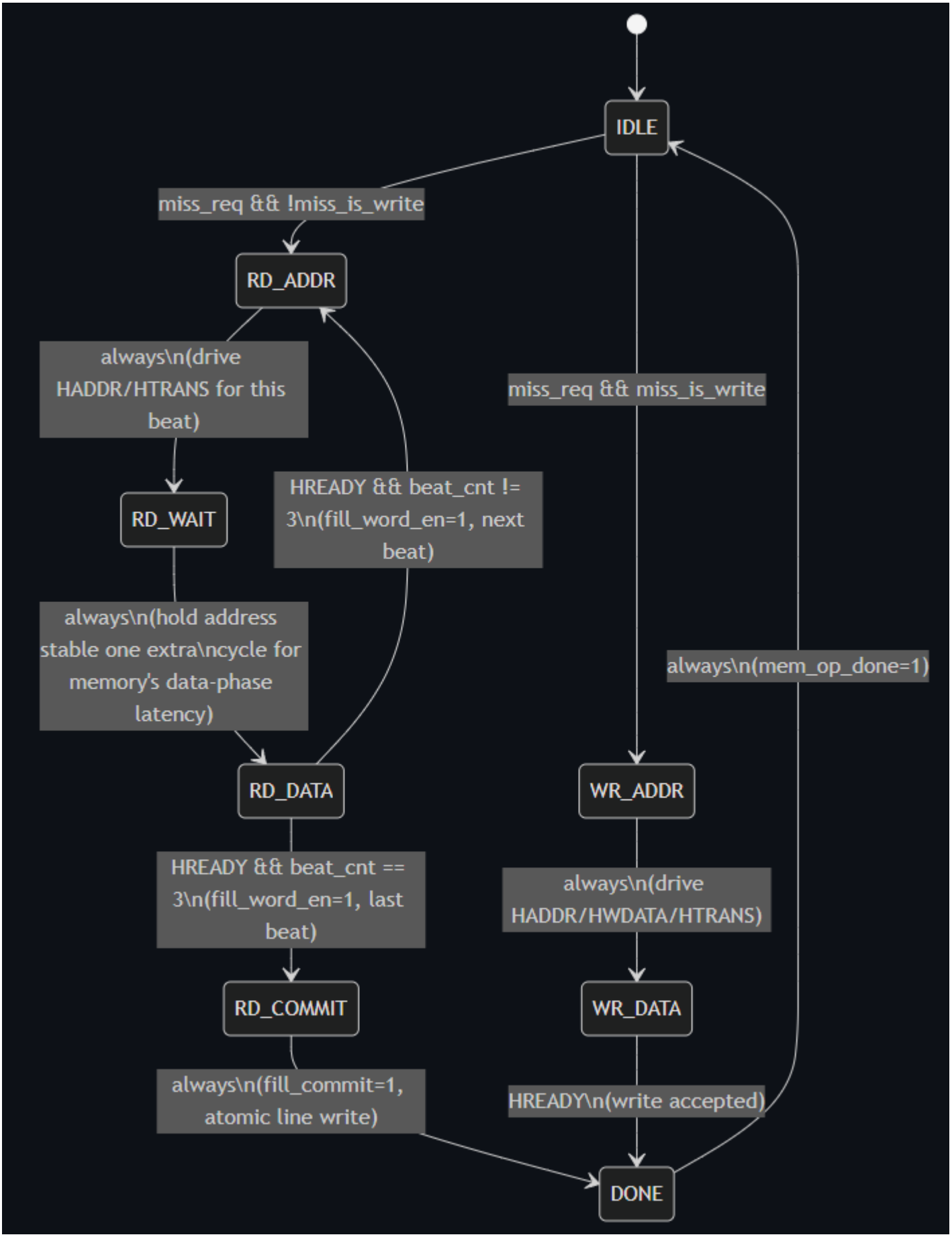
4.6 mem_side_master_fsm.v — “The negotiator with memory”



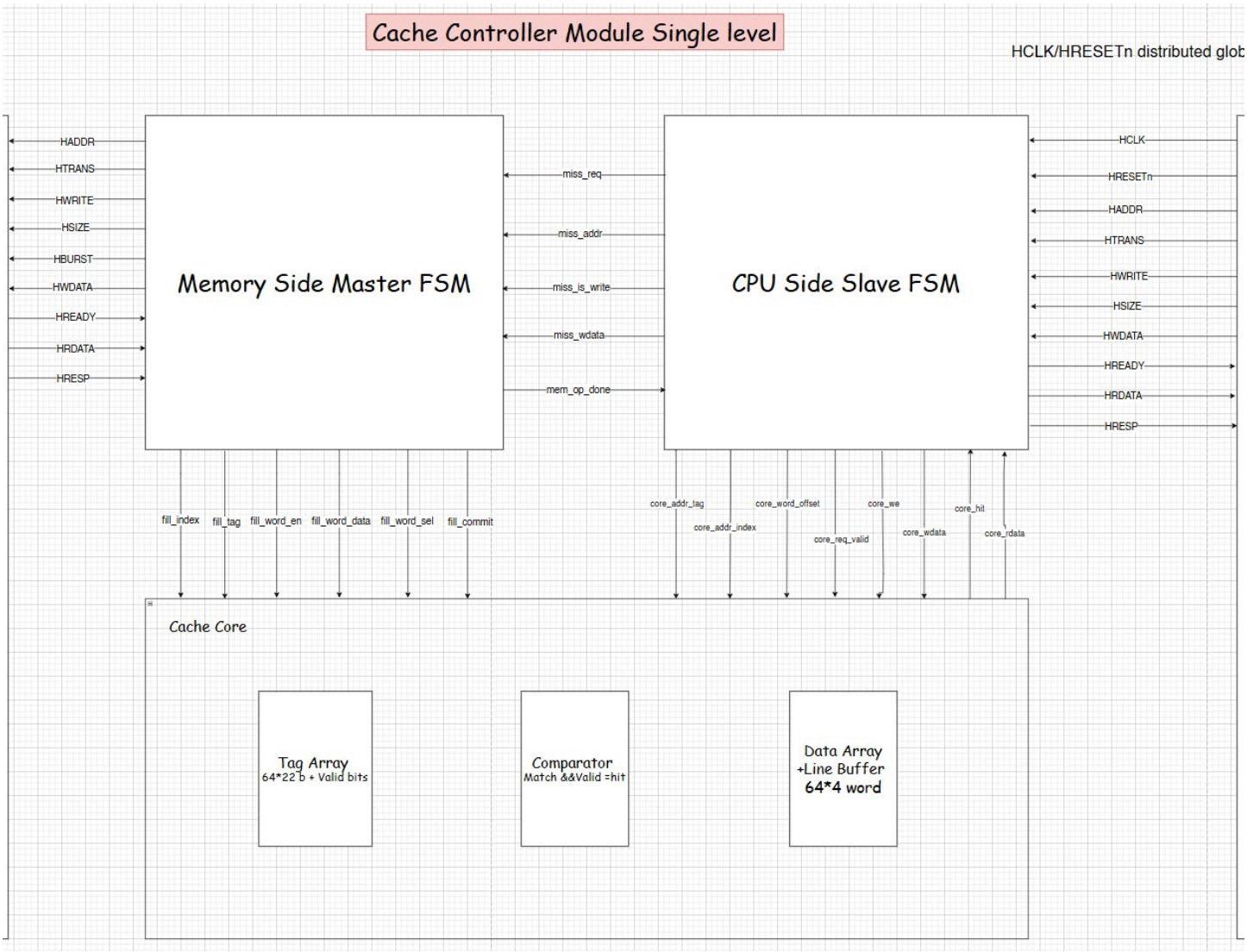
This is the AHB-Lite master state machine that talks to backing memory on the cache's behalf. It only ever does two things: service a read-miss by fetching an entire 4-word line, or service a write-through by writing a single word.

On a read-miss, it issues a 4-beat INCR burst starting at the line-aligned base address, and as each word comes back from memory it feeds it into cache_core's fill-word interface one beat at a time. After the fourth word has arrived, it pulses fill_commit, which is the signal that atomically writes the completed line into the data array and marks the tag valid. On a write-through, it simply issues one AHB write beat with the address and data handed to it by the CPU-side FSM, then signals completion.

State	Meaning
IDLE	Waiting for miss_req; decides read-miss vs. write-through path.
RD_ADDR / RD_WAIT / RD_DATA	Drive the address phase and wait for each of the 4 burst beats' data to become valid.
RD_COMMIT	All 4 words collected; pulse fill_commit to atomically write the line.
WR_ADDR / WR_DATA	Drive a single AHB write beat for the write-through.
DONE	Pulse mem_op_done for one cycle to signal completion back to the CPU-side FSM.


IDLE
miss_req && !miss_is_write
RD_ADDR
always\n(drive HADDR/HTRANS for this beat)
RD_WAIT
always\n(hold address stable one extra\ncycle for memory's data-phase latency)
RD_DATA
HREADY && beat_cnt == 3\n(fill_word_en=1, last beat)
RD_COMMIT
always\n(fill_commit=1, atomic line write)
HREADY && beat_cnt != 3\n(fill_word_en=1, next beat)
miss_req && miss_is_write
WR_ADDR
always\n(drive HADDR/HWDATA/HTRANS)
WR_DATA
HREADY\n(write accepted)
always\n(mem_op_done=1)
DONE

4.7 cache_controller_top.v — "The deliverable, as one reusable block"



This is the top-level integration module and the actual deliverable IP. It wires cpu_side_slave_fsm, cache_core, and mem_side_master_fsm together internally, and exposes exactly two clean interfaces at its boundary: an AHB-Lite slave port for the CPU side, and an AHB-Lite master port for the memory side. All of the internal buses described in Section 3.2 are hidden inside this module; nothing about tags, indices, fill buffers, or internal handshakes is visible from outside it.

Deliberately, no backing memory is instantiated inside this module. The memory-side AHB-Lite master port is brought all the way out to the top level so that any real memory controller or interconnect can be attached later without touching this RTL — keeping the deliverable a genuinely reusable, standalone component rather than something entangled with its own test environment.

4.8 Clock and reset distribution

HCLK and HRESETn are treated as globally and synchronously distributed to every sequential block in the design (tag_valid_array, data_line_array, cpu_side_slave_fsm, mem_side_master_fsm).

5. Verification Summary

Testbench	Scope	Checks	Result
tb_cache_core.v	cache_core in isolation (no AHB)	24	PASS — 0 failures
tb_full_chain.v	Full system via direct block instantiation.	10	PASS — 0 failures
tb_cache_controller_top.v	Standalone top-level module via external AHB ports	8	PASS — 0 failures

Submodules Testing:

1- AHB lite transaction

```
#
# ========================================
# RESET RELEASED
# ========================================
#
#
# ========== TEST 1: READ ==========
# T=25000 | HADDR=00000040 | HTRANS=10 | HWRITE=0 | HWDATA=00000000 | HRDATA=00000000 | HREADY=1
# TEST 1 FAIL: Expected CCCC_0010, Got 00000000
#
# ========== TEST 2: WRITE ==========
# T=35000 | HADDR=00000040 | HTRANS=00 | HWRITE=0 | HWDATA=00000000 | HRDATA=00000000 | HREADY=1
# T=45000 | HADDR=00000040 | HTRANS=10 | HWRITE=1 | HWDATA=deadbeef | HRDATA=cccc0010 | HREADY=1
# TEST 2 FAIL: Expected DEADBEEF, Got cccc0010
#
# ========== TEST 3: READ BACK ==========
# T=55000 | HADDR=00000040 | HTRANS=00 | HWRITE=0 | HWDATA=deadbeef | HRDATA=cccc0010 | HREADY=1
# T=65000 | HADDR=00000040 | HTRANS=10 | HWRITE=0 | HWDATA=deadbeef | HRDATA=cccc0010 | HREADY=1
# TEST 3 FAIL: Expected DEADBEEF, Got cccc0010
#
# ========== TEST 4: SEQUENTIAL READS ==========
# T=75000 | HADDR=00000040 | HTRANS=00 | HWRITE=0 | HWDATA=deadbeef | HRDATA=cccc0010 | HREADY=1
# T=85000 | HADDR=00000040 | HTRANS=10 | HWRITE=0 | HWDATA=deadbeef | HRDATA=deadbeef | HREADY=1
# WORD 0 PASS: deadbeef
# T=95000 | HADDR=00000044 | HTRANS=11 | HWRITE=0 | HWDATA=deadbeef | HRDATA=deadbeef | HREADY=1
# WORD 1 FAIL: Expected CCCC_0011, Got deadbeef
# T=105000 | HADDR=00000048 | HTRANS=11 | HWRITE=0 | HWDATA=deadbeef | HRDATA=deadbeef | HREADY=1
# WORD 2 FAIL: Expected CCCC_0012, Got cccc0011
# T=115000 | HADDR=0000004c | HTRANS=11 | HWRITE=0 | HWDATA=deadbeef | HRDATA=cccc0011 | HREADY=1
# WORD 3 FAIL: Expected CCCC_0013, Got cccc0012
#
# ========== TEST 5: AHB SIGNALS ==========
# HREADY PASS
# HRESP PASS: OKAY
# T=125000 | HADDR=0000004c | HTRANS=00 | HWRITE=0 | HWDATA=deadbeef | HRDATA=cccc0012 | HREADY=1
# T=135000 | HADDR=0000004c | HTRANS=00 | HWRITE=0 | HWDATA=deadbeef | HRDATA=cccc0013 | HREADY=1
#
# ========================================
# ALL TESTS COMPLETED
# ========================================
```

2 – Memory side Master FSM

```
# ========================================
#  MEM SIDE MASTER FSM TESTBENCH
# ========================================
# T=35000 | HTRANS=00 | HADDR=00000000 | HWRITE=0 | HREADY=1 | HRDATA=00000000 | fill_en=0 | sel=0 | commit=0 | done=0
#
# ========================================
#  TEST: READ MISS
#  Address = 00000040
# ========================================
# T=45000 | HTRANS=00 | HADDR=00000000 | HWRITE=0 | HREADY=1 | HRDATA=00000000 | fill_en=0 | sel=0 | commit=0 | done=0
# T=55000 | HTRANS=10 | HADDR=00000040 | HWRITE=0 | HREADY=1 | HRDATA=00000000 | fill_en=0 | sel=0 | commit=0 | done=0
# READ WORD 0: HADDR=00000040 HTRANS=10
# T=65000 | HTRANS=10 | HADDR=00000040 | HWRITE=0 | HREADY=1 | HRDATA=10000010 | fill_en=0 | sel=0 | commit=0 | done=0
# T=75000 | HTRANS=10 | HADDR=00000040 | HWRITE=0 | HREADY=1 | HRDATA=10000010 | fill_en=1 | sel=0 | commit=0 | done=0
# READ WORD 1: HADDR=00000040 HTRANS=10
# T=85000 | HTRANS=11 | HADDR=00000044 | HWRITE=0 | HREADY=1 | HRDATA=10000010 | fill_en=0 | sel=1 | commit=0 | done=0
# T=95000 | HTRANS=11 | HADDR=00000044 | HWRITE=0 | HREADY=1 | HRDATA=10000011 | fill_en=0 | sel=1 | commit=0 | done=0
# READ WORD 2: HADDR=00000044 HTRANS=11
# T=105000 | HTRANS=11 | HADDR=00000044 | HWRITE=0 | HREADY=1 | HRDATA=10000011 | fill_en=1 | sel=1 | commit=0 | done=0
#   -> fill_word_en=1
#   -> fill_word_sel=1
#   -> fill_word_data=10000011
# T=115000 | HTRANS=11 | HADDR=00000048 | HWRITE=0 | HREADY=1 | HRDATA=10000011 | fill_en=0 | sel=2 | commit=0 | done=0
# READ WORD 3: HADDR=00000048 HTRANS=11
# T=125000 | HTRANS=11 | HADDR=00000048 | HWRITE=0 | HREADY=1 | HRDATA=10000012 | fill_en=0 | sel=2 | commit=0 | done=0
# T=135000 | HTRANS=11 | HADDR=00000048 | HWRITE=0 | HREADY=1 | HRDATA=10000012 | fill_en=1 | sel=2 | commit=0 | done=0
# T=145000 | HTRANS=11 | HADDR=0000004c | HWRITE=0 | HREADY=1 | HRDATA=10000012 | fill_en=0 | sel=3 | commit=0 | done=0
# T=155000 | HTRANS=11 | HADDR=0000004c | HWRITE=0 | HREADY=1 | HRDATA=10000013 | fill_en=0 | sel=3 | commit=0 | done=0
# T=165000 | HTRANS=11 | HADDR=0000004c | HWRITE=0 | HREADY=1 | HRDATA=10000013 | fill_en=1 | sel=3 | commit=0 | done=0
#   -> fill_commit = 1
# T=175000 | HTRANS=00 | HADDR=00000000 | HWRITE=0 | HREADY=1 | HRDATA=10000013 | fill_en=0 | sel=0 | commit=1 | done=0
#   -> mem_op_done = 1
# READ MISS PASSED
# T=185000 | HTRANS=00 | HADDR=00000000 | HWRITE=0 | HREADY=1 | HRDATA=10000013 | fill_en=0 | sel=0 | commit=0 | done=1
#
# ========================================
#  TEST: READ MISS
#  Address = 00000080
# ========================================
# T=195000 | HTRANS=00 | HADDR=00000000 | HWRITE=0 | HREADY=1 | HRDATA=10000013 | fill_en=0 | sel=0 | commit=0 | done=0
# T=205000 | HTRANS=10 | HADDR=00000080 | HWRITE=0 | HREADY=1 | HRDATA=10000013 | fill_en=0 | sel=0 | commit=0 | done=0
# READ WORD 0: HADDR=00000080 HTRANS=10
# T=215000 | HTRANS=10 | HADDR=00000080 | HWRITE=0 | HREADY=0 | HRDATA=10000020 | fill_en=0 | sel=0 | commit=0 | done=0
# T=225000 | HTRANS=10 | HADDR=00000080 | HWRITE=0 | HREADY=0 | HRDATA=10000020 | fill_en=0 | sel=0 | commit=0 | done=0
# T=235000 | HTRANS=10 | HADDR=00000080 | HWRITE=0 | HREADY=1 | HRDATA=10000020 | fill_en=1 | sel=0 | commit=0 | done=0
#   -> fill_word_en=1
```

3 – CPU side Slave FSM

```
VSIM 14> run -all
#
# ================ TEST 1: READ HIT ================
#
# T=25000 | STATE=0 | HADDR=00000040 | HTRANS=10 | HWRITE=0 | HREADY=1 | HRDATA=12345678 | core_hit=1 | miss_req=0 | miss_write=0 | done=0
# TEST 1 PASS: Read hit returned correct data
# T=35000 | STATE=0 | HADDR=00000040 | HTRANS=00 | HWRITE=0 | HREADY=1 | HRDATA=00000000 | core_hit=1 | miss_req=0 | miss_write=0 | done=0
#
# ================ TEST 2: READ MISS ================
#
# T=45000 | STATE=0 | HADDR=00000100 | HTRANS=10 | HWRITE=0 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=0 | done=0
# Read miss detected correctly
# T=55000 | STATE=1 | HADDR=00000100 | HTRANS=10 | HWRITE=0 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=0 | done=0
# T=65000 | STATE=1 | HADDR=00000100 | HTRANS=10 | HWRITE=0 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=0 | done=0
# CPU correctly waits for memory
# T=75000 | STATE=1 | HADDR=00000100 | HTRANS=10 | HWRITE=0 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=0 | done=1
# T=85000 | STATE=3 | HADDR=00000100 | HTRANS=10 | HWRITE=0 | HREADY=1 | HRDATA=12345678 | core_hit=0 | miss_req=0 | miss_write=0 | done=0
# TEST 2 completed
#
# ================ TEST 3: WRITE HIT ================
#
# T=95000 | STATE=0 | HADDR=00000200 | HTRANS=10 | HWRITE=1 | HREADY=0 | HRDATA=00000000 | core_hit=1 | miss_req=1 | miss_write=1 | done=0
# TEST 3 PASS: Write hit starts write operation
# T=105000 | STATE=2 | HADDR=00000200 | HTRANS=10 | HWRITE=1 | HREADY=1 | HRDATA=00000000 | core_hit=1 | miss_req=1 | miss_write=1 | done=1
# T=115000 | STATE=0 | HADDR=00000200 | HTRANS=00 | HWRITE=1 | HREADY=1 | HRDATA=00000000 | core_hit=1 | miss_req=0 | miss_write=1 | done=0
#
# ================ TEST 4: WRITE MISS ================
#
# T=125000 | STATE=0 | HADDR=00000300 | HTRANS=10 | HWRITE=1 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=0 | done=0
# TEST 4 PASS: Write miss detected
# T=135000 | STATE=1 | HADDR=00000300 | HTRANS=10 | HWRITE=1 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=0 | done=1
# T=145000 | STATE=3 | HADDR=00000300 | HTRANS=10 | HWRITE=1 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=1 | done=0
# TEST 4 completed
# T=155000 | STATE=2 | HADDR=00000300 | HTRANS=00 | HWRITE=1 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=1 | done=0
# T=165000 | STATE=2 | HADDR=00000300 | HTRANS=00 | HWRITE=1 | HREADY=0 | HRDATA=00000000 | core_hit=0 | miss_req=1 | miss_write=1 | done=0
#
# ================ ALL TESTS DONE ================
#
# ** Note: $finish    : D:/nti_tech/cache_controller/cpu_side/tb_cpu_side_slave_fsm.v(255)
#    Time: 170 ns  Iteration: 0  Instance: /tb_cpu_side_slave_fsm
```

4 – Cache Core

```
# ==== cache_core testbench start ====
#
# -- Test 1: cold-start miss on tag=0, index=0 --
# [PASS] T1: cold-start must be a MISS : 0
#
# -- Test 2: fill index=0 then check all 4 words hit --
# [PASS] T2: word0 hit : 1
# [PASS] T2: word0 data : 0xaaaaa0000
# [PASS] T2: word1 hit : 1
# [PASS] T2: word1 data : 0xaaaaa1111
# [PASS] T2: word2 hit : 1
# [PASS] T2: word2 data : 0xaaaaa2222
# [PASS] T2: word3 hit : 1
# [PASS] T2: word3 data : 0xaaaaa3333
#
# -- Test 3: index-conflict miss (same index, different tag) --
# [PASS] T3: different tag same index must MISS : 0
#
# -- Test 4: write-hit partial-word integrity --
# [PASS] T4: write-hit reports hit : 1
# [PASS] T4: word1 now updated : 0xdeadbeef
# [PASS] T4: word0 unchanged (neighbor integrity) : 0xaaaaa0000
# [PASS] T4: word2 unchanged (neighbor integrity) : 0xaaaaa2222
# [PASS] T4: word3 unchanged (neighbor integrity) : 0xaaaaa3333
#
# -- Test 5: independent line at index=5 --
# [PASS] T5: index=5 cold miss (unaffected by index=0 activity) : 0
# [PASS] T5: index=5 hit after its own fill : 1
# [PASS] T5: index=5 word2 data : 0xbbbbb2222
# [PASS] T5: index=0 still hits (unaffected by index=5 fill) : 1
# [PASS] T5: index=0 word0 still correct : 0xaaaaa0000
#
# -- Test 6: req_valid gating --
# [PASS] T6: hit must be 0 when req_valid=0 : 0
#
# -- Test 7: write-miss at index=10, tag=0x7 --
# [PASS] T7: write to unfilled line must MISS : 0
# [PASS] T7: write-hit after fill : 1
# [PASS] T7: word now holds written value : 0xcafef00d
#
# ==== SUMMARY: 24 checks run, 0 failed ====
# ==== ALL TESTS PASSED ====
# ** Note: $finish    : D:/nti_tech/cache_controller/cache_core/tb_cache_core.v(312)
#    Time: 556 ns  Iteration: 0  Instance: /tb_cache_core
```

5 – Cache Controller top

```
# ==== cache_controller_top wrapper testbench start ====
#
# -- Test 1: cold read at addr=0x40 (expect miss then correct data) --
# [PASS] T1: read data after miss-fill : 0xcccc0010
#
# -- Test 2: re-read same address (expect fast hit) --
# [PASS] T2: read data on hit matches : 0xcccc0010
#
# -- Test 3: write-hit then read-back --
# [PASS] T3: read-back after write-hit : 0xdeadbeef
#
# -- Test 4: write-through reached backing memory --
# [PASS] T4: memory array updated by write-through : 0xdeadbeef
#
# -- Test 5: second independent line at addr=0x80 --
# [PASS] T5: second line reads correct data : 0xcccc0020
# [PASS] T5: first line still holds written value : 0xdeadbeef
#
# -- Test 6: back-to-back transfer immediately after completion --
# [PASS] T6: third line reads correct data : 0xcccc0030
# [PASS] T6: back in IDLE (HREADY=1 with no pending req) : 1
#
# ==== SUMMARY: 8 checks run, 0 failed ====
# ==== ALL TESTS PASSED ====
```

6. RTL and Verification File Manifest

File	Type	Description
tag_valid_array.v	RTL	Tag storage and valid bits for all 64 lines
comparator.v	RTL	Combinational hit/miss decision logic
data_line_array.v	RTL	128-bit-wide data storage, read/write-hit/fill logic
cache_core.v	RTL	Integrates the three blocks above into one protocol-agnostic cache
cpu_side_slave_fsm.v	RTL	AHB-Lite slave FSM facing the CPU
mem_side_master_fsm.v	RTL	AHB-Lite master FSM facing backing memory
cache_controller_top.v	RTL (deliverable top)	Full integration; exposes AHB-Lite slave + master ports
ahb_mem_model.v	Verification only	Behavioral backing-memory stand-in, fixed 1-cycle latency
tb_cache_core.v	Testbench	Unit-level self-checking testbench for cache_core
tb_full_chain.v	Testbench	Full-chain integration testbench
tb_cache_controller_top.v	Testbench	Top-level module verification via external AHB ports

Note: there is a testbench for each separate module also to make the functional testing.

7. Golden Reference Model for the Cache Controller

The reference model is a from-scratch Python implementation of the cache's architectural specification: the address breakdown, the direct-mapped storage organization, and the write-through-with-write-allocate-on-miss policy. Critically, it was written directly from the specification documented in the project report, not by translating or copying logic from the Verilog RTL. This independence is what gives the cross-check its value: if a bug existed in the RTL's interpretation of the spec, there would be no reason for an independently written model to reproduce the same bug.

Two verification activities were performed with this model:

- Replay of the exact CPU transaction sequences used in tb_full_chain.v / tb_cache_controller_top.v, compared against the same expected constants those testbenches already assert (and pass) under Icarus Verilog.
- A randomized stress test of 2,500 read/write operations across five seeds, checking an invariant that goes beyond the RTL testbenches' hand-picked vectors: that backing memory always ends up holding exactly what a direct, uncached memory system would hold, regardless of cache hit/miss history.

Result: 12 out of 12 replayed checks passed, and 0 mismatches were found across all 2,500 randomized operations. The golden model agrees with the RTL's verified behavior in every case tested.

7.1 Why Build a Golden Reference Model?

A self-checking testbench (like tb_full_chain.v) is written by the same engineer, using the same mental model of the design, as the RTL itself. This is normal and necessary, but it has a structural blind spot: if the engineer misunderstood the specification in a particular way, that same misunderstanding can leak into both the RTL and the testbench's expected values, and the mismatch would never be caught — the testbench would pass while quietly checking the wrong thing.

7.2 What the Model Represents, and How It Maps to the RTL

The reference model collapses the RTL's three cooperating blocks — cpu_side_slave_fsm, cache_core, and mem_side_master_fsm — into a single functional unit, because at the transaction level a CPU issuing a read or write does not observe the internal hand-off between these blocks; it only observes the final result. The table below shows the correspondence.

RTL Concept	Reference Model Equivalent	Notes
cpu_side_slave_fsm hit/miss decision	CacheModel._lookup()	Same tag comparison + valid-bit check as comparator.v
cache_core (tag_valid_array + comparator + data_line_array)	CacheLine dataclass + CacheModel.lines[]	64-entry list of CacheLine objects, one per index
mem_side_master_fsm read-miss 4-beat burst fill	CacheModel._fill_line()	Fetches all 4 words at once (no burst timing modeled)
mem_side_master_fsm single-beat write-through	CacheModel.do_write() memory-write step	One word written directly to BackingMemory
ahb_mem_model.v backing memory + 0xCCCC_0000+i init pattern	BackingMemory class	Same initialization formula, for numerically comparable results
cpu_side_slave_fsm STATE_COMPLETE (re-issue write after fill)	do_write()'s fill-then-write-hit sequence	Models the write-allocate deviation explicitly
HADDR tag/index/word-offset slicing	split_address()	Same bit positions: [31:10] / [9:4] / [3:2]

7.3 Results

7.3.1 Replay Against RTL-Verified Expected Values

Running compare_with_rtl.py produced the following:

Check	Model Result	RTL-Verified Expected	Status
T1: read data after miss-fill	0xcccc0010	0xcccc0010	PASS
T2: read data on hit matches	0xcccc0010	0xcccc0010	PASS
T3: read-back after write-hit	0xdeadbeef	0xdeadbeef	PASS
T4: memory updated by write-through	0xdeadbeef	0xdeadbeef	PASS
T5: second line reads correct data	0xcccc0020	0xcccc0020	PASS
T5: first line still holds written value	0xdeadbeef	0xdeadbeef	PASS
T6: third line reads correct data	0xcccc0030	0xcccc0030	PASS
T7: read data on new line 0x140	0xcccc0050	0xcccc0050	PASS
T7: exactly one fill occurred	1	1	PASS
T7: re-read after fill is stable	0xcccc0050	0xcccc0050	PASS
WA: write-miss read-back reflects write	0xcafef00d	0xcafef00d	PASS
WA: write-through reached backing memory	0xcafef00d	0xcafef00d	PASS

Grand total: 12 checks, 0 failed. The golden model agrees with every RTL-testbench-derived expected value it was checked against.

7.3.2 Randomized Invariant Stress Test

stress_test.py exercises a property the hand-written testbenches never explicitly check: after any arbitrary, unplanned sequence of reads and writes, does backing memory end up holding exactly what a direct (uncached) memory system would hold? This must be true for any correct write-through design, regardless of which lines happened to be hit or missed along the way.

Five seeded runs of 500 random operations each (2,500 operations total) were executed, comparing the model's cache-fronted memory against a plain, cache-free BackingMemory instance driven with the identical operation sequence:

Seed	Ops	Read Mismatches	Final-Memory Mismatches	Fills	Write-Throughs	Result
1	500	0	0	64	254	PASS
2	500	0	0	64	260	PASS
3	500	0	0	64	265	PASS
42	500	0	0	64	243	PASS
12345	500	0	0	64	251	PASS

All five seeds independently reach 64 fills — one per distinct cache line, out of the 256-word / 64-line address space used in the model — confirming that every line was exercised across the random sequences. Zero mismatches occurred across all 2,500 operations.

8. Future Work

- L2 cache / cache hierarchy (explicitly deferred, not in current scope).
- HRESP error responses (SPLIT/RETRY/ERROR) for a more complete AHB-Lite compliance profile.
- Support for sub-word (byte/halfword) access via HSIZE.
- Formal reset synchronizer / clock-domain-crossing support if the design is integrated into a multi-clock SoC.
- Gate-level synthesis and STA closure as a follow-on to this RTL-level deliverable.

9. Conclusion

The direct-mapped cache controller described in this report meets its defined scope: a single-level, write-through cache with clean AHB-Lite interfaces on both the CPU and memory sides, implemented in synthesizable Verilog and verified at both the unit and full-system level.

10. Source Codes:

https://github.com/tawfeek202/Direct-Mapped-Cache-Controller-with-an-AHB-Lite-Interface